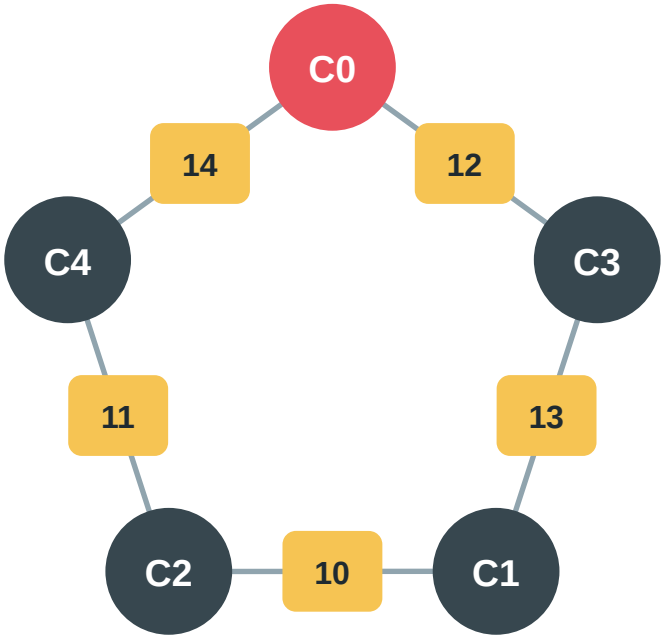
Children in a Circle

- Each child has 20 features (yes / no)
- Difference = number of different features (Hamming distance)
- Children stand in a circle: the last one is next to the first one
- Goal: the order with the biggest total difference between neighbours



Best circle for 5 children: 12+13+10+11+14 = 60

Starting point: the code from class (Child, fitness, normalize, tweak).

Setup: imports and constants

```
from __future__ import annotations
from collections.abc import Sequence
from functools import total_ordering
from random import sample, seed
from itertools import permutations
from math import factorial

NUM_CHILDREN = 5
NUM_CARACTERISTICS = 20
seed(42)  # same random children at every run
```

The Child class (from class)

```
@total_ordering
class Child:
    _name: str
    _caracteristics: tuple[bool, ...]
    def __init__(self, name: str, caracteristics: Sequence[bool]):
        self._name = name
        self._caracteristics = tuple(caracteristics)
    # Hamming distance: number of characteristics that are different
    def difference(self, other: Child) -> int:
        return sum(s != o for s, o in zip(self._caracteristics, other._caracteristics))
    def __str__(self) -> str:
        return f"{self._name}:{''.join('+' if c else '-' for c in self._caracteristics)}"
    def __repr__(self) -> str:
        return f"{self._name}"
    # comparison based on id(), needed by normalize
    def __eq__(self, other):
        return id(self) == id(other)
    def __lt__(self, other):
        return id(self) < id(other)
```

Create the 5 children

```
# create the children with random characteristics
CHILDREN = list()
for c in range(NUM_CHILDREN):
    name = f"Child{c}"
    CHILDREN.append(
        Child(
            name,
            sample(
                [True, False], k=NUM_CARACTERISTICS, counts=[NUM_CARACTERISTICS, NUM_CARACTERISTICS]
            ),
        )
    )
for c in CHILDREN:
    print(c)
```

Output

```
Child0:++++++++--++--++----
Child1:--+-+++--+++--+++-+-
Child2:-++--+-++--+-+-+--+-
Child3:++++-+--++---+---++-
Child4:+-+---++----++--++++
```

Difference between every pair

```
# difference between every pair of children
for a in CHILDREN:
    for b in CHILDREN:
        if a._name < b._name:
            print(a._name, "-", b._name, ":", a.difference(b))
```

Output

```
Child0 - Child1 : 7
Child0 - Child2 : 9
Child0 - Child3 : 12
Child0 - Child4 : 14
Child1 - Child2 : 10
Child1 - Child3 : 13
Child1 - Child4 : 13
Child2 - Child3 : 7
Child2 - Child4 : 11
Child3 - Child4 : 10
```

Fitness: the score of a circle

```
# sum of the differences between neighbours (each pair counted twice)
def fitness(solution: Sequence[Child]) -> int:
    sol = [CHILDREN[0]] + list(solution)  # Child0 is always the first
    fitness = 0
    for cp, c, cn in zip(sol, sol[1:] + sol[0:1], sol[2:] + sol[0:3]):
        fitness += c.difference(cp)
        fitness += c.difference(cn)
    return fitness
```

normalize and tweak

```
# a circle and its reverse are the same: always keep the same one
def normalize(solution: Sequence[Child]) -> list[Child]:
    if solution[0] > solution[-1]:
        return list(solution[::-1])
    else:
        return list(solution)
# swap two random children
def tweak(solution: Sequence[Child]) -> list[Child]:
    i1, i2 = sample(range(len(solution)), k=2)
    new_solution = list(solution)
    new_solution[i1] = solution[i2]
    new_solution[i2] = solution[i1]
    return normalize(new_solution)
```

My change: normalize compares the first and the LAST child (in class: the first and the second).

Example: fitness of one circle

```
# example: fitness of the circle Child0-Child1-Child2-Child3-Child4
solution = CHILDREN[1:]
circle = [CHILDREN[0]] + list(solution)
total = 0
for i in range(len(circle)):
    a = circle[i]
    b = circle[(i + 1) % len(circle)]  # the last child is next to the first one
    print(a._name, "-", b._name, ":", a.difference(b))
    total += a.difference(b)
print("sum of the differences:", total)
print("fitness:", fitness(solution), "(= 2 x sum, each pair is counted twice)")
```

Output

```
Child0 - Child1 : 7
Child1 - Child2 : 10
Child2 - Child3 : 7
Child3 - Child4 : 10
Child4 - Child0 : 14
sum of the differences: 48
fitness: 96 (= 2 x sum, each pair is counted twice)
```

Examples: tweak and normalize

```
# example: tweak
print("before:", solution, fitness(solution))
new_solution = tweak(solution)
print("after: ", new_solution, fitness(new_solution))
```

Output

```
before: [Child1, Child2, Child3, Child4] 96
after:  [Child3, Child2, Child1, Child4] 112
```

```
# example: a solution and its reverse give the same list
s = list(CHILDREN[1:])
print(s, "->", normalize(s))
print(s[::-1], "->", normalize(s[::-1]))
print("same fitness:", fitness(s), fitness(s[::-1]))
```

Output

```
[Child1, Child2, Child3, Child4] -> [Child1, Child2, Child3, Child4]
[Child4, Child3, Child2, Child1] -> [Child1, Child2, Child3, Child4]
same fitness: 96 96
```

Two solutions

1 Exhaustive search

- Try every order, keep the best
- Always finds the best answer
- Only for few children: (n-1)! orders

2 Hill climbing

- Random start, tweak = swap two children
- Keep the new order if it is not worse
- Fast, but can stop in a local optimum

120 = 120

5 children: hill climbing finds the same best score as exhaustive search

24 → 10^{30}

orders to try with 5 children and with 30 children

624 → 816

30 children: random order vs hill climbing with 5 random restarts

Exhaustive search: best = 120

```
# exhaustive search: try every permutation and keep the best
best_solution = None
best_fitness = 0
for solution in permutations(CHILDREN[1:]):
    print(solution, fitness(solution))
    if fitness(solution) > best_fitness:
        best_solution = solution
        best_fitness = fitness(solution)
print()
print("BEST:", best_solution, best_fitness)
```

Output

```
...
(Child4, Child3, Child2, Child1) 96

BEST: (Child3, Child1, Child2, Child4) 120
```

Hill climbing reaches the same 120

```
# hill climbing: start from a random solution
solution = normalize(sample(CHILDREN[1:], k=NUM_CHILDREN - 1))
print("start:", solution, fitness(solution))
for step in range(100):
    new_solution = tweak(solution)
    if fitness(new_solution) > fitness(solution):
        print("step", step, ":", new_solution, fitness(new_solution), "better!")
    # keep the new solution if it is not worse
    if fitness(new_solution) >= fitness(solution):
        solution = new_solution
print("final:", solution, fitness(solution))
print("exhaustive search:", best_fitness)
```

Output

```
start: [Child1, Child3, Child2, Child4] 104
step 0 : [Child4, Child1, Child3, Child2] 112 better!
step 4 : [Child3, Child1, Child2, Child4] 120 better!
final: [Child3, Child1, Child2, Child4] 120
exhaustive search: 120
```

Number of solutions: (n-1)!

```
# number of solutions with n children: (n-1)!
for n in [5, 8, 10, 15, 20, 30]:
    print(n, "children:", factorial(n - 1), "solutions")
```

Output

```
5 children: 24 solutions
8 children: 5040 solutions
10 children: 362880 solutions
15 children: 87178291200 solutions
20 children: 1216451004088320000 solutions
30 children: 884176199373970195454361600000 solutions
```

30 children: create them

```
# 30 children: too many solutions to try them all, so we use only hill climbing
NUM_CHILDREN = 30
CHILDREN = list()
for c in range(NUM_CHILDREN):
    name = f"Child{c}"
    CHILDREN.append(
        Child(
            name,
            sample(
                [True, False], k=NUM_CARACTERISTICS, counts=[NUM_CARACTERISTICS, NUM_CARACTERISTICS]
            ),
        )
    )
random_solution = normalize(sample(CHILDREN[1:], k=NUM_CHILDREN - 1))
print("a random solution:", fitness(random_solution))
```

Output

```
a random solution: 624
```

30 children: 5 random restarts

```
# hill climbing with 5 random restarts
results = []
for run in range(5):
    solution = normalize(sample(CHILDREN[1:], k=NUM_CHILDREN - 1))
    for step in range(5_000):
        new_solution = tweak(solution)
        if fitness(new_solution) >= fitness(solution):
            solution = new_solution
    print("run", run + 1, ":", fitness(solution))
    results.append(fitness(solution))
print("best:", max(results))
print("upper bound (every pair differs on every characteristic):",
      2 * NUM_CHILDREN * NUM_CARACTERISTICS)
```

Output

```
run 1 : 804
run 2 : 800
run 3 : 816
run 4 : 796
run 5 : 812
best: 816
upper bound: 1200
```